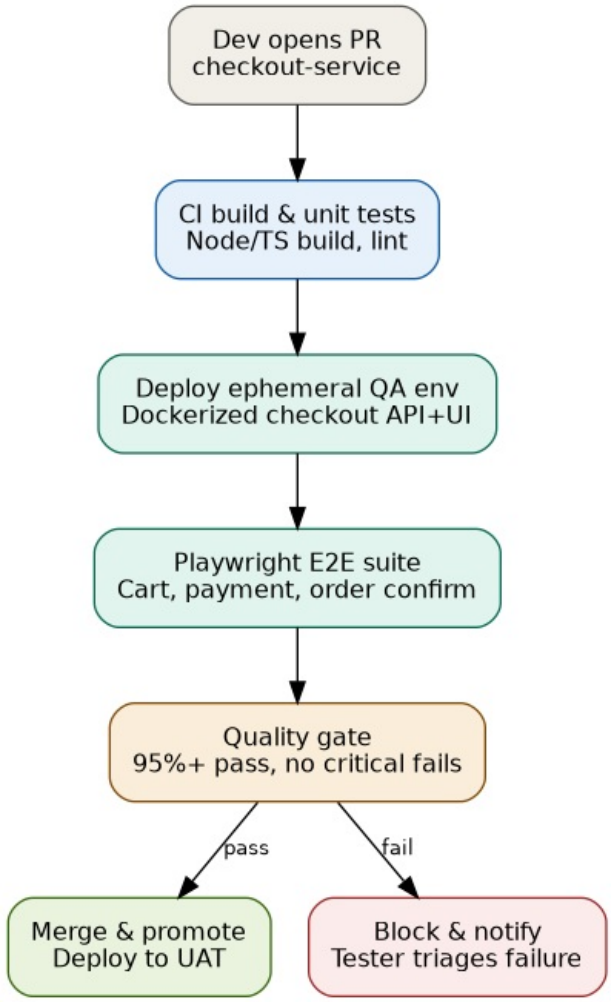
GitHub Actions & Continuous Testing — Notes for Automation Testers

Real-time scenario: checkout flow for an e-commerce app

A team ships a `checkout-service` (cart, payment, order confirmation) for a retail web app. Every code change must pass automated tests before it reaches UAT/PROD. As the automation tester, you own the E2E suite and its place in the pipeline.

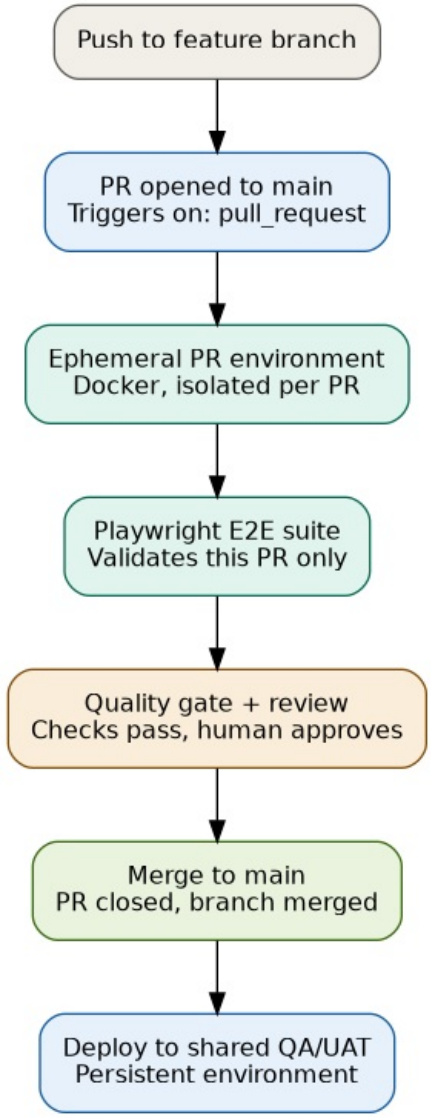
Pipeline flow



Checkout automation pipeline

PR flow vs merge flow — don’t conflate these

Opening a PR does **not** mean the code has merged anywhere. The PR is a request to merge, sitting pending until it passes checks and review. There are two distinct pipelines and two distinct environments involved:



PR to merge environment flow

- **Ephemeral, per-PR environment** — spun up just to validate *this* change, torn down after the run. Prevents two PRs’ test data/deployments from colliding.
- **Shared, persistent QA/SAT/UAT environment** — exists continuously, updated by a *separate* pipeline that fires only after merge to `main`. This is where broader integration testing and release sign-off happens — closer to a multi-region release-management workflow.

Stage-by-stage: what the automation tester does

1. **PR opened** — Feature branch code (not `main`) is checked out; this triggers the workflow (`on: pull_request`). Nothing has merged yet.
2. **CI build & unit tests** — Not your suite, but if this fails your E2E job never runs. Know how to read these logs to tell "build broke" apart from "my tests broke."
3. **Ephemeral QA environment** — You own the Docker Compose / environment config that spins up a throwaway checkout API + UI per PR, so tests never collide with another run's data.
4. **Playwright E2E suite** — Your core responsibility. Test cases for:
   - Add to cart, update quantity, remove item
   - Apply/remove discount code
   - Payment: valid card, declined card, expired card
   - Order confirmation page and confirmation email trigger
     Sharded across parallel jobs (matrix strategy) to keep the run under ~10 minutes.
5. **Quality gate** — You define what "pass" means: e.g. 95%+ pass rate and zero failures in payment-critical tests. This is wired as a **required status check**, so GitHub blocks the merge button until it's green.
6. **Pass → merge & promote** — Build auto-deploys to UAT. No manual regression pass needed.
7. **Fail → block & notify** — PR is blocked, Slack/Teams alert fires, you triage: real bug, flaky test, or environment issue.

Sample GitHub Actions workflow

```yaml
name: checkout-e2e
on:
  pull_request:
    paths: ['services/checkout/**']

jobs:
  e2e:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        shard: [1, 2, 3, 4]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: docker compose -f docker-compose.qa.yml up -d
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shard }}/4
        env:
          BASE_URL: http://localhost:4000
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report-${{ matrix.shard }}
          path: playwright-report/
```

- **Trigger scoped with `paths:`** — only runs when checkout code actually changes, saving CI minutes.

- **Matrix + shard** — splits the suite across 4 parallel runners.
- **`if: always()`** — uploads the report even when tests fail, so failures are debuggable.
- **Required check** — configured in branch protection rules, not in this file, referencing the `e2e` job name.

Knowledge checklist (what to actually know, not just recite)

- ☐ Difference between `push`, `pull_request`, `schedule`, `workflow_dispatch`, `workflow_call` triggers
- ☐ How `strategy: matrix` + `--shard` parallelizes a suite
- ☐ `actions/cache` to skip reinstalling `node_modules` / browser binaries every run
- ☐ Secrets vs GitHub Environments (QA/UAT/PROD) with required approvals
- ☐ `actions/upload-artifact` for reports, screenshots, videos on failure
- ☐ Branch protection + required status checks — the real mechanism behind a "quality gate"
- ☐ `concurrency`: to cancel stale runs when a new commit lands on the same PR
- ☐ Flaky test handling: retries (`retries:` in Playwright config), quarantine tags, tracking flake rate
- ☐ Reusable/composite workflows (`workflow_call`) to avoid duplicating YAML across repos

How to talk about it in an interview

Use a before → action → after structure, tied to a real number where possible:

> "Our checkout regression relied on a manual pass before every release. I built a Playwright suite covering cart, payment, and order confirmation, wired it into GitHub Actions as a required check on every PR touching the checkout service, and sharded it across 4 runners to keep the run under 10 minutes. A failing run now blocks the merge automatically instead of surfacing in UAT — which cut our release verification time and caught issues before they reached a shared environment."

This invites natural follow-ups (flaky tests, what the gate threshold was, how environments were isolated) — which is exactly where the checklist above pays off.